

forge-auditAgent

Requirements Specification — Modular Agentic AI Environment

Requirements Document v1.0

July 2026

Problem & Scope

Problem Statement

The challenge: Jupyter Notebooks are the de facto standard for data science and ML, yet they introduce systemic risks:

- Untrusted execution — no deterministic guarantee of safety or correctness
- No structured audit protocol — existing code review misses notebook-specific failure modes
- Data leakage through preprocessing ordering
- Cross-validation contamination
- Inseparable training/inference logic
- Hidden global state across cells

Goal: A modular, local-first generative AI agent environment for notebook construction and audit.

System Overview

forge-auditAgent operates on two integrated tracks:

Construction Workbench

1. **Scaffold** — section templates, dependency pinning, seed config
2. **Write** — incremental authoring, explicit state
3. **Validate** — kernel restart, cell idempotency

Audit Engine (6 passes)

1. Structural Overview
2. Reproducibility Check
3. Data Integrity Review
4. ML Correctness Audit
5. Code Quality Review
6. Deployment Readiness

Both powered by a **provider-agnostic LLM orchestration layer** (llama.cpp, Ollama, OpenAI) under the LLM-as-a-Judge paradigm.

System Architecture

Hardware Architecture (Current Desktop)

Component	Specification
CPU	x86_64 / ARM64
RAM	≥ 8 GB (recommended)
GPU	Optional: Metal, CUDA, ROCm

Execution flow:

User Machine

- Flet Desktop App (test/prompts/)
 - Hardware Detection
 - Model Download (HuggingFace Hub)
 - Local LLM Server (llama-cpp-python)
 - Audit Pipeline (6 passes)
 - Export (JSON / MD / PDF)
-

Software Architecture

Layer	Components
GUI (Flet)	Hardware, Models, Server tabs
	Benchmark, Settings tabs
	Audit tab: Loader → Pipeline → Export
LLM Layer	llama-cpp-python (in-process)
	Ollama API (local)
	OpenAI API (cloud, opt-in)
Persistence	Engram memory, JSON config, reports

Audit pipeline flow:

Notebook → Structural → Reproducibility
Pass 1 Pass 2

→ Data Integrity → ML Correctness → Code Quality → Deployment

Functional Requirements

Functional Requirements — Loading and Input (RF-01 to RF-04)

ID	Description	Acceptance Criteria
RF-01	Load and parse .ipynb files from local filesystem; validate JSON structure, cell types, and metadata.	Valid → Notebook(valid=True); malformed → valid=False with errors.
RF-02	Fetch notebooks from GitHub raw URLs; auto-convert github.com/blob/ to raw.githubusercontent.com/.	Normalised URL resolves; parsed notebook matches expected structure.
RF-03	Scan local notebooks directory and present files in a dropdown (extensions: .ipynb, .py, .md, .txt).	All matching files found; first valid notebook auto-loaded.
RF-04	Guide notebook authoring through 3-phase Construction Workbench (Scaffold → Write → Validate).	Each phase visually distinct; canonical 8-section header order enforced.

Functional Requirements — Audit Engine (RF-05 to RF-11)

ID	Description	Acceptance Criteria
RF-05	Execute six sequential audit passes, each with structured deliverable and risk score (L/M/H).	All passes run; each returns <code>PassResult</code> with required fields.
RF-06	Pass 1: Section map, red flags, focus-area scope.	Headers identified; broken imports, orphaned cells flagged.
RF-07	Pass 2: Dependency pinning, seeds, hardcoded paths, re-executability.	Risk score computed; dependency file and seed calls verified.
RF-08	Pass 3: Train/test split ordering, leakage, missing data, ingestion validation.	Split-before-preprocessing enforced; leakage patterns flagged.
RF-09	Pass 4: Metric appropriateness, CV integrity, hyperparameter boundaries, baseline.	Checklist produced; CV leakage detected; baseline required.
RF-10	Pass 5: Repetitive code (≥ 3), dead code, naming, output hygiene.	Smell report; repetition threshold enforced; bloated outputs flagged.
RF-11	Pass 6: Artifact export, separability, resource docs, environment, privacy.	Risk score; artifact versioning and separation verified.

Functional Requirements — LLM and Export (RF-12 to RF-18)

ID	Description	Acceptance Criteria
RF-12	Hybrid evaluation: deterministic checks for objective criteria; LLM judging for subjective.	Deterministic checks run without API calls; LLM routes through configured provider.
RF-13	Support three LLM providers (llama.cpp, Ollama, OpenAI) interchangeable via strategy pattern.	Each returns valid evaluation for identical prompts; switching does not require restart.
RF-14	Real-time audit progress via callback or SSE; one event per completed pass.	Callback fires after each pass; UI updates within 500 ms.
RF-15	Export reports in JSON, Markdown, and PDF formats.	Each export produces a valid file at the specified path.
RF-16	Notebook DB scanner listing and loading files from <code>test/prompts/notebooks/</code> .	Populates dropdown on click; first valid file auto-loads.
RF-17	Accept manual local file paths and load valid notebooks.	Path + “Load Local” loads notebook or returns clear error.
RF-18	Full offline operation for local-only operations.	Core functions work without internet; cloud LLM is opt-in.

Non-Functional Requirements

Non-Functional Requirements (RNF-01 to RNF-10)

ID	Description	Acceptance Criteria
RNF-01	Cross-platform: Linux (WSL, native), macOS, Windows; GPU: Metal, CUDA, ROCm.	Launches and core features work on all three platforms.
RNF-02	Start to interactive within 5 s (≥ 8 GB RAM, SSD).	Measured from <code>python main.py</code> to responsive UI.
RNF-03	Local LLM response ≤ 30 s for ≤ 7 B models on CUDA GPU.	Mean over 10 audit runs ≤ 30 s.
RNF-04	Graceful handling of missing optional deps; no startup crash.	Starts with features disabled; no traceback.
RNF-05	Full 6-pass audit for ≤ 50 cells in ≤ 120 s (deterministic + 7B local model).	End-to-end from Run Audit click.
RNF-06	All user data stays local by default; zero egress without explicit action.	No outbound requests during local operations.
RNF-07	No UI freezes beyond 200 ms during audit execution.	Frame drops measured during pipeline run.
RNF-08	LLM provider extensible via Open/Closed Principle.	New provider = new class implementing <code>LLMProvider</code> .
RNF-09	All audit results persisted for later review.	Reports accessible after restart;

Verification Plan

Verification Plan (TC-01 to TC-06)

ID	Req	Objective / Procedure
TC-01	RF-01	Load valid and malformed notebooks; verify valid=True/False with appropriate errors.
TC-02	RF-02	Enter <code>github.com/user/repo/blob/main/test.ipynb</code> ; verify URL normalisation and load.
TC-03	RF-16	Click Scan DB; verify dropdown populated and first notebook auto-loaded.
TC-04	RF-05	Load sample notebook, select all focus areas, run audit; verify 6 result cards.
TC-05	RF-12	Run audit with local LLM; verify deterministic checks pass without API calls.
TC-06	RF-15	After audit, export JSON; verify valid file at expected path.

Verification Plan (TC-07 to TC-13)

ID	Req	Objective / Procedure
TC-07	RF-15	After audit, export PDF; verify valid PDF with formatted report.
TC-08	RF-17	Type path to .ipynb in Local path, click Load Local; verify load and cell count.
TC-09	RF-18	Disconnect network; verify core functions work, cloud features show unavailable.
TC-10	RNF-04	Run on machine without GPU libs; verify app starts gracefully.
TC-11	RNF-06	Run full local audit while monitoring network; verify zero outbound connections.
TC-12	RNF-09	Restart app; verify all previous audit reports present and readable.
TC-13	RNF-10	Run full audit monitoring RAM via psutil; verify ≤ 4 GB above baseline.

Justification

Justification

Criterion	Compliance
Problem statement	Notebook reproducibility crisis and lack of structured audit clearly identified with concrete failure modes.
System cohesion	Construction Framework, Audit Engine, and LLM Orchestration form a closed feedback loop: build → audit → improve.
Measurable requirements	All RF/RNF quantified with specific acceptance criteria; IDs cross-referenced to verification tests.
Architecture clarity	Hardware and software diagrams document current implementation; strategy pattern ensures extensibility.
Verification plan	13 test cases covering all functional and key non-functional requirements with procedures and priorities.
Local-first design	All core operations run offline with zero data egress; cloud LLM is strictly opt-in.

forge-auditAgent

Requirements Specification v1.0

Modular Agentic AI Environment — July 2026